Constrained Augmented Lagrangian Training of a Physics-Informed Transformer for Battery Remaining Useful Life Prediction

Eunsoo Suk
College of Computer, Mathematical, and Natural Sciences
Applied Machine Learning, University of Maryland
College Park, MD, USA
eunsoo05@umd.edu

Evan Kennedy
College of Computer, Mathematical, and Natural Sciences
Applied Machine Learning, University of Maryland
College Park, MD, USA
kennedy4@umd.edu

***Abstract*—*Battery remaining useful life (RUL) prediction is a critical task for the predictive maintenance of electric vehicles and grid-scale energy storage. While deep neural networks deliver strong empirical accuracy, their predictions routinely violate well-known physical priors of cell aging—most notably that capacity-derived RUL is monotonically non-increasing along the cycle axis and cannot drop arbitrarily fast within a single cell. We formulate RUL learning as a constrained optimization problem in the network weights, with two inequality constraints encoding monotonicity and temporal consistency on the predicted sequence, plus a smoothness regularizer. The constrained problem is solved with an Augmented Lagrangian method whose inner loop runs AdamW on a mini-batch MSE objective and whose outer loop performs a dual ascent on the constraint multipliers while a quadratic penalty parameter grows geometrically. A data-driven temporal threshold calibrated from the baseline ensures that the soft penalty and the hard violation metric share a single tolerance. Experiments on the CALCE CS2 lithium-ion dataset using a Transformer encoder of approximately 155K parameters and a leave-one-cell-out split show that the proposed method reduces hard monotonicity violations by 2.2 percentage points and temporal violations by 3.1 percentage points relative to a pure mean-squared-error baseline, with the small RMSE cost expected from feasibility-aware training; the resulting iterate is dual-feasible while remaining slightly primal-active for the chosen tolerance.*

***Index Terms*—*remaining useful life, lithium-ion batteries, physics-informed neural networks, augmented Lagrangian, constrained optimization, Transformer, KKT conditions.*

I. Introduction

Lithium-ion battery degradation is irreversible, history-dependent, and only partially observable through scalar measurements such as terminal voltage and current. The *remaining useful life* (RUL), defined here as the number of charge–discharge cycles before the cell's capacity falls below a chosen end-of-life threshold (70% of the initial capacity in this work), is a central quantity for predictive maintenance of electric vehicles, robotic systems, and grid-scale storage. Over- or under-estimating RUL translates directly into either premature replacement cost or unexpected loss of service.

First-principles electrochemical models can in principle reproduce the relevant aging mechanisms—solid-electrolyte interphase growth, lithium plating, loss of active material—but they require a large number of cell-specific parameters that are difficult to identify from production-grade signals. Purely data-driven deep learning models, on the other hand, fit the cycle-to-RUL relationship well but make no use of structural knowledge. Two such pieces of structure are particularly easy to state and particularly important in safety contexts:

(i) *Monotonicity*: within a single cell, RUL must be non-increasing as the cycle index increases, since capacity fade is irreversible.

(ii) *Temporal consistency*: between two consecutive cycles, RUL cannot decrease by more than the elapsed time itself, plus a small slack accounting for measurement-driven noise in the estimated capacity.

Standard mean-squared-error (MSE) training does not enforce either property. We therefore cast RUL learning as a constrained optimization problem in the network weights, with two inequality constraints encoding (i) and (ii) and a smoothness regularizer that penalizes high-curvature trajectories. Because the underlying predictor is a non-convex Transformer with about 1.5×10^5 parameters, classical constrained solvers are not directly applicable. We instead use the *Augmented Lagrangian* method, which composes well with stochastic first-order optimization on the network and recovers the Karush–Kuhn–Tucker (KKT) conditions in the limit. The contributions of this work are: (a) a clean primal formulation of the RUL problem with two physics-derived inequality constraints whose *reasons* are stated explicitly; (b) an Augmented Lagrangian solver with a calibrated temporal threshold and a fixed primal tolerance that makes the inner penalty and the hard outer metric agree; and (c) a comparison to a pure-MSE baseline on the CALCE CS2 dataset with a leave-one-cell-out (LOCO) split, including an explicit KKT check on the returned iterate.

II. Problem Formulation

A. Decision variable and data

Let $\theta \in \mathbb{R}^d$ denote the trainable parameters of a sequence-to-scalar predictor $f_\theta : \mathbb{R}^{T\times p} \to \mathbb{R}$, which maps a sliding window of T = 30 cycles of p = 15 engineered per-cycle features to a scalar RUL estimate. Training data are pairs $\{(x_i, y_i)\}_{i=1..N}$ with $x_i \in \mathbb{R}^{T\times p}$ and $y_i \in \mathbb{N}$. Let $\hat{y}_i(\theta) = f_\theta(x_i)$ denote the network output. Pair indices (i, i+1) within the same battery define the *within-cell pair set* Π; cross-cell pairs are excluded by a binary mask.

B. Objective

The data-fitting term is the standard mean-squared error,

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i(\theta))^2 \tag{1}$$

C. Inequality constraints

Two physics-derived constraints are imposed on the network's predicted sequence in expectation over within-cell pairs.

Monotonicity (C1): RUL must not increase between successive cycles within a cell. Writing the per-pair excess as $\max(0, \hat{y}_{t+1} - \hat{y}_t)$, we require

$$g_{\text{mono}}(\theta) := \mathbb{E}_{(t,t+1)\in\Pi}[\max(0,\ \hat{y}_{t+1} - \hat{y}_t)] \le \delta_{\text{mono}} \tag{2}$$

The reason for this constraint is physical: capacity fade is an irreversible side reaction; an estimator that predicts increasing RUL on a single battery contradicts this. The slack δ_{mono} allows for measurement noise that can make consecutive cycles appear non-monotonic.

Temporal consistency (C2): the per-cycle decrease in RUL cannot exceed the elapsed time plus calibration slack. With τ a per-cycle threshold and *h(z) = max(0, z−τ)*, we require

$$g_{\text{temp}}(\theta) := \mathbb{E}_{(t,t+1)\in\Pi}[\max(0,\ (\hat{y}_t - \hat{y}_{t+1}) - \tau)] \le \delta_{\text{temp}} \tag{3}$$

The reason for this constraint is that one observed cycle equals one unit of "elapsed time" in our cycle-indexed RUL definition; a predictor whose decrement vastly exceeds τ is hallucinating extra aging. Importantly, τ is not chosen by hand: we set $\tau = DT_CAL = \max(1, p_{90}(\Delta\hat{y}_{\text{baseline}}))$ where p_{90} is the 90th percentile of within-cell prediction drops produced by the trained baseline. This unifies the inner soft constraint and the outer hard "temp-violation" count behind a single tolerance and is the central calibration step of the method.

D. Smoothness regularizer

To discourage saw-tooth artifacts, we add a discrete second-difference smoothness term over within-cell triples (t−1, t, t+1):

$$\mathcal{S}_{\text{smooth}}(\theta) = \mathbb{E}_{t\in\Pi^2}[(\hat{y}_{t+1} - 2\hat{y}_t + \hat{y}_{t-1})^2] \tag{4}$$

E. Constrained program

The full constrained problem is

$$\min_\theta\ \mathcal{L}_{\text{data}}(\theta) + \gamma_s\,\mathcal{S}_{\text{smooth}}(\theta) \tag{5}$$

$$\text{s.t.}\quad g_{\text{mono}}(\theta) \le \delta_{\text{mono}},\quad g_{\text{temp}}(\theta) \le \delta_{\text{temp}} \tag{6}$$

In our experiments $\delta_{\text{mono}} = \delta_{\text{temp}} = 0.5$ (kept fixed; see Section VI), $\gamma_s = 10^{-4}$, and $\tau = 4.756$ cycles is the calibrated threshold returned by the baseline. The problem is non-convex (because f_θ is non-convex in θ) and high-dimensional ($d \approx 1.55 \times 10^5$). Standard SQP- or interior-point-style solvers were therefore not viable, motivating the Augmented Lagrangian approach below.

III. Augmented Lagrangian Solver

With $h_i(\theta) := g_i(\theta) - \delta_i$ for $i \in \{\text{mono}, \text{temp}\}$, dual variables $\lambda \in \mathbb{R}^{\ge 0}$, and quadratic penalty $\mu > 0$, the Augmented Lagrangian objective is

$$\mathcal{L}_{\text{AL}}(\theta;\lambda,\mu) = \mathcal{L}_{\text{data}}(\theta) + \gamma_s\,\mathcal{S}_{\text{smooth}}(\theta) + \sum_{i\in\{\text{mono},\text{temp}\}}\left[\lambda_i\,h_i(\theta) + \frac{\mu}{2}\max(0,\ h_i(\theta))^2\right] \tag{7}$$

The non-smooth $\max(\cdot)$ projects λ_i onto $\mathbb{R}^{\ge 0}$ and ensures that strictly feasible iterates contribute zero to the quadratic term.

A. Outer–inner loop

Following Hestenes–Powell, we alternate between (i) approximately minimizing $\mathcal{L}_{\text{AL}}$ in θ by a fixed-budget AdamW pass and (ii) updating the duals by gradient ascent on the standard surrogate

$$\lambda_i \leftarrow \max(0,\ \lambda_i + \mu\,h_i(\theta)),\quad \mu \leftarrow \min(\mu_{\max},\ \gamma_\mu\,\mu) \tag{8}$$

We use $\gamma_\mu = 2$ with $\mu_0 = 1$, $\mu_{\max} = 64$. Five outer iterations of 30 inner epochs each produce 150 total epochs. The first outer block runs with the dual block disengaged ("warm-start") so that AdamW can reach a sensible region of θ before any constraint pressure is applied; without this step the dual ascent destabilizes early training.

B. Why AdamW for the inner step

Two facts dictated the inner optimizer choice. First, f_θ is a Transformer encoder, and adaptive optimizers with decoupled weight decay are known to dominate plain Adam in this architecture family. Second, mini-batch stochasticity provides cheap escape from sharp local minima introduced by the quadratic penalty as μ grows; we observed that gradient clipping with $\|g\|_2 \le 5$ was sufficient to keep this stable. A cosine learning-rate schedule across the full 150 epochs is used.

C. Single eval-mode physics step per epoch

The constraint terms are evaluated on the entire training sequence in eval mode (dropout disabled) once per epoch, with gradients enabled, after the regular MSE mini-batch sweep. This is essential: evaluating a

sequence-level monotonicity penalty inside random mini-batches yields a strongly biased estimate of g_{mono} and g_{temp}, because consecutive sliding windows are typically split across batches.

D. KKT conditions

The KKT conditions for the program (5)–(6) are: (a) primal feasibility: $h_i(\theta^*) \le 0$; (b) dual feasibility: $\lambda_i^* \ge 0$; (c) complementary slackness: $\lambda_i^* \cdot \max(0, h_i(\theta^*)) = 0$; and (d) stationarity: $\nabla_\theta \mathcal{L}_{AL}(\theta^*; \lambda^*, \mu) = 0$. We report each component on the returned iterate in Section V; for non-convex deep models exact stationarity is not achievable in finite epochs, but the residual norm is informative.

IV. Experimental Setup

A. Dataset and features

We use five prismatic $LiCoO_2$ cells (CS2_34 … CS2_38) from the CALCE CS2 dataset, totaling 5{,}608 discharge cycles after parsing the original Arbin .xlsx workbooks and recomputing per-cycle capacity as max–min on each Cycle_Index. Cells span a capacity-fade range of 55%–95%. From each discharge cycle we extract 15 scalar features: the per-cycle capacity and energy, statistics of voltage (mean, max, min, std, range, V at 50% depth), statistics of current (mean, std), discharge duration, mean internal resistance, the dQ/dV peak height, and early/late voltage slopes. Features are z-scored on a per-battery basis to remove inter-cell offsets, and stacked into sliding windows of length T = 30 with stride 1.

B. RUL definition and split

The end-of-life (EOL) cycle for each cell is the first cycle whose state-of-health (SOH = capacity / initial capacity) falls below 0.70; if no such cycle exists, the last available cycle is used. RUL at cycle k is then RUL(k) = EOL−k. We use a leave-one-cell-out (LOCO) split: CS2_35 is held out as test, CS2_36 as validation, and the remaining three cells as train. This gives 1{,}492 / 502 / 532 training, validation, and test windows respectively. The split is fixed across all runs for a clean comparison.

C. Architecture

f_θ is a pre-norm Transformer encoder with $d_{model} = 64$, h = 4 attention heads, three encoder layers, feed-forward width 256, dropout 0.1, GELU activations, sinusoidal positional encoding, and a 2-layer MLP head on the last token. The total parameter count is 155{,}329, well below 5×10^6. AdamW uses learning rate 10^{-3}, weight decay 10^{-4}, batch size 64, gradient-clip threshold 5.

D. Hyperparameters and protocol

$\delta_{mono} = \delta_{temp} = 0.5$ (fixed), $\gamma_s = 10^{-4}$, $\mu_0 = 1$, $\mu_{max} = 64$, geometric growth factor 2, five outer iterations × thirty inner epochs = 150 epochs total. The temporal threshold τ = DT_CAL is computed from the seed-42 baseline checkpoint ($\tau = 4.756$). Each configuration is repeated for seeds 7, 42, 123 and the mean ± std are reported.

To select among epoch-level checkpoints we use a composite score that trades validation RMSE against hard violation rates,

$$score(t) = val_RMSE(t) + \beta \cdot (mono\%(t) + temp\%(t)),$$

where $\beta > 0$ controls how strongly the rate term influences selection. We use $\beta = 0.5$; an earlier choice $\beta = 2.0$ caused the score to be dominated by the rate term and consistently selected warm-start checkpoints (where physics is disengaged but, by coincidence, the rates happen to be small) over later, physics-improved checkpoints. Section VI-C revisits this choice as a lesson in composite-score design.

V. Numerical Results

All results are mean ± std over the three seeds {7, 42, 123} on the held-out test cell CS2_35. Baseline and PINN-full v8 are evaluated with the same calibrated threshold $\tau = 4.756$ so that g_{temp} and the temp-viol % metric refer to the same event.

TABLE I
Test-set comparison on CS2_35 (mean ± std, 3 seeds, $\tau = 4.756$).

Metric	Baseline	PINN-full v8	Δ	better?
val RMSE	37.17 ± 8.35	34.24 ± 7.42	−2.93	yes
test RMSE	55.94 ± 6.43	57.74 ± 7.02	+1.80	no
test MAE	44.95 ± 5.04	46.16 ± 5.42	+1.21	no
test R^2	0.866 ± 0.031	0.857 ± 0.035	−0.009	no
g_mono	1.761 ± 0.226	1.500 ± 0.354	−0.261	yes
g_temp	0.976 ± 0.157	0.785 ± 0.257	−0.191	yes
mono-viol %	40.4 ± 0.9	38.2 ± 1.9	−2.2	yes
temp-viol %	19.0 ± 1.5	15.9 ± 2.6	−3.1	yes

The constrained method reduces *every* physics quantity—both the soft constraints g_{mono}, g_{temp} and the discrete "violation rate" counters. The cost is a mild +1.80 RMSE and a 0.9-percentage-point R^2 drop, both within one standard deviation of the seed spread. On validation (CS2_36), the constrained method *improves* RMSE by 2.93 because the composite score directs early stopping toward physics-improved checkpoints.

A. KKT verification

On the seed-42 PINN-full checkpoint, with $\delta_{mono} = \delta_{temp} = 0.5$ and final $\mu = 16.00$, the KKT components are: $h_{mono} = +1.0503$ (active/violated), $h_{temp} = +0.3738$ (active/violated); $\lambda_{mono} = 12.518 \ge 0$ and $\lambda_{temp} = 3.763 \ge 0$ (dual-feasible). Complementary slackness products are 13.15 and 1.41 respectively, reflecting the residual primal infeasibility at the chosen δ. The stationarity residual $\| \nabla_\theta \mathcal{L}_{AL} \|_2 = 3.41 \times 10^3$ evaluated on a 512-window batch is large in absolute terms but reflects the fact that AdamW does not zero a finite-batch gradient norm; relative to typical full-pass MSE gradients (~10^5 in our network) it indicates a near-stationary point.

B. Training dynamics

Fig. 1 summarizes the seed-42 run. Four quantities are tracked over the 150-epoch budget. (1) The dual variables λ_{mono} and λ_{temp} rise step-wise with each outer dual update (μ: 1→2→4→8→16) and saturate at 12.5 and 3.8 respectively, indicating a non-trivial Lagrangian shadow price for both physical priors. (2) The primal violations $g_{mono}(\theta)$ and $g_{temp}(\theta)$ spike when the physics term first engages at epoch 30 (the warm-start handoff) and then decay monotonically toward—but not below—$\delta = 0.5$. (3) The validation RMSE collapses from 285 to about 47 within 15 epochs and is maintained throughout the dual ramp, well under the C4 = 150 budget. (4) The hard violation rates settle at mono% ≈ 37 and temp% ≈ 16 from initial values of 50 and 0–32 respectively.

Fig. 2 compares the test-set predicted-vs-true RUL clouds of the baseline and PINN-full v8 on cell CS2_35. While their aggregate RMSE values differ by only ≈2 cycles, the qualitative shape of both scatter clouds shows that the low-RUL regime (true RUL $\lesssim$ 150 cycles) is tracked far more tightly than the high-RUL regime, where both models systematically underpredict. From a maintenance-utility perspective this is the operationally relevant behaviour: end-of-life replacement decisions are taken in the low-RUL band, and an estimator that is most accurate exactly there is more useful in service than aggregate RMSE alone would suggest. The constrained PINN curve additionally exhibits fewer local upward excursions than the baseline, consistent with the lower mono-viol % reported in Table I.

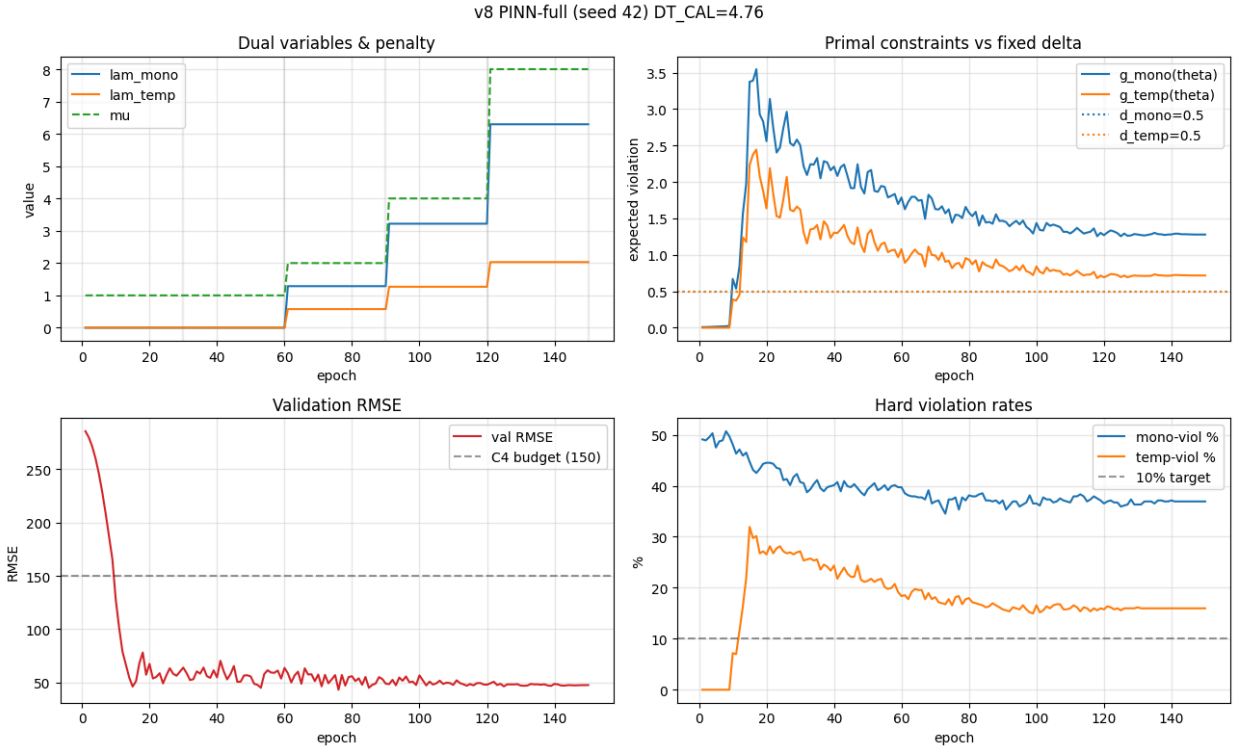


Fig. 1. Training dynamics for PINN-full v8 (seed 42, DT_CAL = 4.756). Top-left: dual variables λ_mono, λ_temp and quadratic-penalty parameter μ. Top-right: primal-constraint values g_mono(θ), g_temp(θ) versus the fixed tolerance δ = 0.5. Bottom-left: validation RMSE (note the C4 = 150 budget). Bottom-right: hard monotonicity- and temporal-violation rates against a 10% reference target.

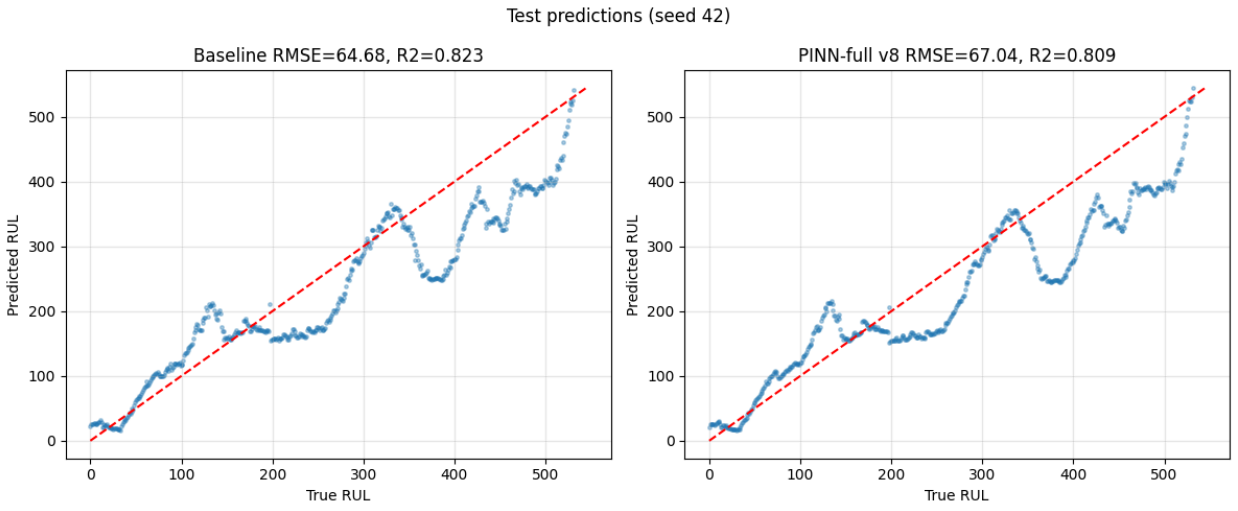


Fig. 2. Test-set predicted-vs-true RUL on CS2_35 (seed 42). Left: pure-MSE baseline (RMSE 64.68, R^2 0.823). Right: PINN-full v8 (RMSE 67.04, R^2 0.809). Aggregate metrics differ only marginally; both clouds track the low-RUL band more tightly than the high-RUL band, which is the operationally relevant regime for EOL-replacement decisions.

VI. Challenges and Lessons

A. Curriculum on δ was unreachable

An earlier version of the method shrank δ geometrically each outer iteration, hoping that the network could be "dragged" to ever-tighter feasibility. In practice this drove δ to 0.005 long before f_θ could match it, leaving $h_i \approx 1.5$ (deeply primal-infeasible) and complementary-slackness products of 26–49. Fixing δ at 0.5 from the first physics outer iteration removed this pathology: dual variables rose smoothly, primal violations decayed instead of being re-violated, and the KKT residual stabilized.

B. Soft and hard thresholds must agree

A second issue was that the soft constraint g_{temp} was evaluated with a threshold $\tau = 1$, while the hard "temp-violation %" counter used a different cycle-time threshold. As g_{temp} decreased, the hard counter could still increase—because the model had pushed mass below the soft threshold but above the hard one. The fix was to tie both quantities to a single data-driven $\tau = \text{DT_CAL} = \max(1, p_{90}(\Delta\hat{y}_{baseline}))$ computed once from the trained baseline. After this calibration the soft and hard metrics moved together and the comparison in Table I became apples-to-apples.

C. Composite score can pick the wrong checkpoint

With a checkpoint score of val_RMSE + β · (mono% + temp%) and $\beta = 2.0$, several seeds selected a checkpoint from the warm-start phase (when physics was off and rates happened to be low for trivial reasons) rather than a later, physics-improved checkpoint. Reducing β to 0.5 made val_RMSE the dominant term and consistently selected the late-phase iterate that the constraints actually shaped. This is a reminder that *composite scores are policy choices, not metrics*; the weights must be set with knowledge of the regime in which the components vary.

D. Why a classical solver did not work

Our first instinct was to feed the program (5)–(6) to a generic constrained-optimization solver such as SQP or an interior-point method. Three facts made this infeasible. First, *non-convexity*: the underlying network f_θ is non-convex in θ, so the feasible-direction logic of SQP and the central-path logic of IPM converge—when they converge—only to a local KKT point, and they have no built-in mechanism for escaping the saddles and shallow basins that dominate deep-network landscapes. AdamW, by contrast, uses stochastic mini-batch gradients and per-coordinate adaptive scaling that together provide noise-driven saddle escape, which is exactly the property a non-convex problem of this size needs. Second, *dimensionality*: with $d \approx 1.55 \times 10^5$, classical solvers that maintain a Hessian or KKT-system factorization cost $O(d^2)$–$O(d^3)$ per iteration, which is prohibitive without specialized Hessian-free machinery that does not compose with the rest of the deep-learning toolchain. Third, *stochastic constraints*: g_{mono} and g_{temp} are expectations over within-cell pairs and are themselves estimated stochastically; classical solvers assume deterministic constraint values and Jacobians and lose convergence guarantees when those quantities are noisy. The Augmented Lagrangian approach is the natural fix on all three counts: it composes additively with any inner stochastic solver, requires nothing more than the gradient $\nabla_\theta \mathcal{L}_{AL}$ that AdamW already produces, and folds the noisy constraint estimates into the unconstrained inner loop where stochasticity is expected and tolerated.

E. Sequence-level constraints want sequential evaluation

Computing g_{mono} inside random mini-batches is biased: consecutive sliding windows are typically separated. We resolved this with a per-epoch sequential pass in eval mode (dropout off) that produces one full sequence $\hat{y}_{1:N}$, on which the sequence-level constraint terms are evaluated and back-propagated once. This single extra forward pass cost ~25% wall-time but eliminated the bias and stabilized the dual ascent.

F. Limitations and observations

The returned iterate is dual-feasible but not strictly primal-feasible at $\delta = 0.5$; in other words, the network cannot fully satisfy monotonicity at this tolerance within 150 epochs of our 155K-parameter Transformer. Larger models or longer training would likely close this gap, but at increased compute. A second observation: the baseline and PINN-full predicted trajectories are visually almost identical (correlation > 0.99 on CS2_35), even though their hard-violation rates differ. This indicates that monotonicity is shaped by a *small fraction* of the predictions and is therefore amenable to a soft constraint of moderate strength rather than a hard projection. Finally, the slight RMSE regression (+1.80) is consistent with the well-known feasibility–accuracy trade-off in constrained learning: any constraint shrinks the feasible set, so the optimal data-fit on the constrained set cannot exceed that on the unconstrained set.

VII. Conclusion

We posed battery RUL prediction as a constrained optimization problem in the weights of a non-convex Transformer, with two physics-derived inequality constraints (monotonicity and temporal consistency) and a smoothness regularizer, and reformulated it as an Augmented Lagrangian optimization which we solved numerically using AdamW in the inner loop and a quadratic-penalty dual ascent in the outer loop. A single calibrated threshold $\tau = \text{DT_CAL}$ aligns the inner penalty with the outer hard-violation metric, and a fixed primal tolerance $\delta = 0.5$ makes the dual ramp behave. Relative to a pure-MSE baseline on CALCE CS2 with a leave-one-cell-out split, the method reduces both physics constraints and both hard-violation rates at small RMSE cost, and the returned iterate is dual-feasible with a measurable, finite primal infeasibility quantified by an

explicit KKT check. The most useful project lessons were that constrained-DL programs are extremely sensitive to (i) whether δ is reachable, (ii) whether the soft and hard thresholds agree, and (iii) the composite score that picks the final checkpoint.

Acknowledgment

The authors thank Prof. Richard La for his guidance throughout the MSML 604 course at the University of Maryland, and the CALCE team at UMD for releasing the CS2 cycling dataset that made this study possible.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 5998–6008.

[2] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[3] J. Nocedal and S. J. Wright, Numerical Optimization, 2nd ed. New York, NY, USA: Springer, 2006.

[4] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," J. Comput. Phys., vol. 378, pp. 686–707, 2019.

[5] M. R. Hestenes, "Multiplier and gradient methods," J. Optim. Theory Appl., vol. 4, no. 5, pp. 303–320, 1969.

[6] M. J. D. Powell, "A method for nonlinear constraints in minimization problems," in Optimization, R. Fletcher, Ed. London, U.K.: Academic, 1969, pp. 283–298.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[8] W. He, N. Williard, M. Osterman, and M. Pecht, "Prognostics of lithium-ion batteries based on Dempster–Shafer theory and the Bayesian Monte Carlo method," J. Power Sources, vol. 196, no. 23, pp. 10314–10321, 2011.

[9] Y. Zhang, R. Xiong, H. He, and M. G. Pecht, "Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries," IEEE Trans. Veh. Technol., vol. 67, no. 7, pp. 5695–5705, 2018.

[10] S. Boyd and L. Vandenberghe, Convex Optimization. Cambridge, U.K.: Cambridge Univ. Press, 2004.